

Apostila Teórica: Fundamentos do Back-End e Ecossistema Node.js

Módulo: Programação Back-End (PBE-1) **Projeto:** Aetheria Engine API

1. Arquitetura de Software: A Divisão de Responsabilidades

Na engenharia de software moderna, a construção de sistemas complexos (como um MMORPG online ou um e-commerce) exige a separação clara de responsabilidades. Essa divisão é conhecida como arquitetura **Cliente-Servidor**.

O Cliente (Front-End)

O Front-End é a "vitrine". É tudo aquilo com que o usuário interage diretamente.

- **Em aplicações Web:** O HTML, CSS e o JavaScript que rodam no navegador (Chrome, Edge).
- **Em Games (Ex: Aetheria Online):** É a Unity, Unreal Engine ou o App do celular. Ele carrega os modelos 3D, toca as trilhas sonoras e lê os comandos do teclado.
- **Regra de Ouro:** O Front-End é **inseguro**. Nunca devemos confiar nos dados processados no cliente, pois um usuário mal-intencionado pode alterar o código do Front-End (usando *Cheat Engine* ou ferramentas de desenvolvedor) para enviar ao sistema que ele possui 1.000.000 de moedas de ouro ou HP infinito.

O Servidor (Back-End)

O Back-End é o "cérebro" oculto e a "fortaleza" da aplicação.

- **Responsabilidades:** Ele recebe as requisições do Front-End, valida todas as regras de negócio de forma isolada e segura, comunica-se com o Banco de Dados para buscar ou salvar informações e, por fim, devolve uma resposta.
- **Servidor Autoridade:** Em jogos online, o Back-End é a única fonte da verdade. Se o Front-End diz "Eu tirei 5000 de dano do chefe", o Back-End calcula: "Os status deste jogador não permitem 5000 de dano. Dano rejeitado. Jogador banido".

2. O Ecossistema Node.js

Historicamente, o JavaScript foi criado para rodar **apenas** dentro dos navegadores web (para fazer botões clicáveis e animações simples). Linguagens como Java, C# e PHP dominavam o Back-End.

Em 2009, Ryan Dahl pegou o motor de JavaScript do Google Chrome (chamado **V8 Engine**) e o embutiu em um programa em C++ que podia ser instalado direto no sistema operacional (Windows, Linux, Mac). Assim nasceu o **Node.js**.

O que o Node.js NÃO é:

- Ele não é uma linguagem de programação (a linguagem continua sendo o JavaScript/TypeScript).

- Ele não é um framework.

O que o Node.js É:

- Ele é um **Ambiente de Execução (Runtime Environment)**. Ele permite que o computador entenda JavaScript fora do navegador, podendo ler arquivos do disco rígido, abrir portas de rede, acessar bancos de dados, etc.
-

3. Entendendo o NPM e o `package.json`

Quando construímos um servidor Back-End, não criamos a roda do zero. Precisamos de bibliotecas de terceiros (para criptografar senhas, enviar emails, comprimir imagens, etc).

- **NPM (Node Package Manager):** É o maior repositório de softwares de código aberto do mundo. Ele vem instalado junto com o Node.js e permite baixar pacotes com um único comando no terminal (ex: `npm install express`).
 - **package.json:** É o "RG" e o "Alvará de Funcionamento" do seu projeto. Ele é gerado pelo comando `npm init -y`. Ele contém o nome do seu aplicativo, a versão e, mais importante, a **lista exata de dependências** que o seu projeto precisa para rodar.
 - **Pasta `node_modules`:** Quando você faz um download via NPM, o código da biblioteca de terceiros é salvo nesta pasta. Ela é muito pesada e **nunca deve ser enviada para o GitHub**. Se outro desenvolvedor baixar o seu projeto sem essa pasta, basta ele digitar `npm install` no terminal, e o NPM lerá o `package.json` para baixar tudo automaticamente.
-

4. Por que TypeScript?

O JavaScript é uma linguagem de **Tipagem Dinâmica e Fraca**. Isso significa que uma variável que nasceu como um `Número` pode, a qualquer momento do código, se transformar em uma `String` (texto). No Back-End, onde lidamos com segurança, transações financeiras e lógicas complexas de jogos, essa "liberdade" do JavaScript causa bugs silenciosos e desastrosos.

A Microsoft criou o **TypeScript**, que é um *Superset* (um "superconjunto") do JavaScript.

- **Tipagem Estática:** Com TypeScript, nós declaramos "contratos" rígidos (`let HP: number = 100;`). Se tentarmos guardar texto ali, o programa quebra antes mesmo de ir para o servidor, salvando-nos de erros em produção.
- **A Transpilação:** O Node.js não entende TypeScript. Por isso, usamos um compilador (`tsc`) que traduz (transpila) o nosso código TypeScript seguro para JavaScript puro, para que o Node possa finalmente executá-lo. Durante as aulas, usamos a ferramenta `ts-node` para acelerar esse processo.